

Signature:.....
Date:.....

1. TITLE OF THE LAB REPORT EXPERIMENT

Implement Knapsack Problem and Huffman Coding Algorithm.

2. OBJECTIVES/AIM

Implement and analyze two fundamental algorithms (Knapsack and Huffman Coding) to gain hands-on experience in algorithm design and implementation.

Understand the underlying principles of dynamic programming and greedy algorithms.

Apply these algorithms to real-world problems like resource optimization and data compression.

3. Problem1

Given a list of coins i.e 1 taka, 5 taka and 10 taka, can you determine the total number of combinations of the coins in the given list to make up the number N taka.

4. IMPLEMENTATION_1

```
import java.util.Scanner;
```

```
public class CoinCombinations {  
    public static int countCombinations(int[] coins, int N) {  
        int[] dp = new int[N + 1];  
        dp[0] = 1;  
  
        for (int i = 0; i < coins.length; i++) {  
            for (int j = coins[i]; j <= N; j++) {  
                dp[j] += dp[j - coins[i]];  
            }  
        }  
  
        return dp[N];  
    }  
}
```

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    System.out.print("Enter the target amount (N): ");  
}
```

```

    int N = scanner.nextInt();

    int[] coins = {1, 5, 10};

    int combinations = countCombinations(coins, N);
    System.out.println("The total number of combinations to make " + N + " taka is: " +
combinations);

    scanner.close();
}
}

```

5. OUTPUT_1

```

Enter the target amount (N): 28
The total number of combinations to make 28 taka is: 12

```

6. Problem2

Construct a Huffman tree by using these nodes.

Value	A	B	C	D	E	F
Frequency	5	25	7	15	4	12

4. IMPLEMENTATION_2

```

import java.util.PriorityQueue;

class HuffmanNode implements Comparable<HuffmanNode> {
    char value;
    int frequency;
}

```

```
HuffmanNode left, right;
```

```
HuffmanNode(char value, int frequency) {  
    this.value = value;  
    this.frequency = frequency;  
    this.left = null;  
    this.right = null;  
}
```

```
@Override  
public int compareTo(HuffmanNode other) {  
    return this.frequency - other.frequency;  
}  
}
```

```
public class HuffmanTree {  
  
    public static HuffmanNode buildHuffmanTree(char[] values, int[] frequencies) {  
        PriorityQueue<HuffmanNode> pq = new PriorityQueue<>();  
        for (int i = 0; i < values.length; i++) {  
            pq.add(new HuffmanNode(values[i], frequencies[i]));  
        }  
  
        while (pq.size() > 1) {  
            HuffmanNode left = pq.poll();  
            HuffmanNode right = pq.poll();  
            HuffmanNode newNode = new HuffmanNode('-', left.frequency +  
right.frequency);  
            newNode.left = left;  
            newNode.right = right;  
            pq.add(newNode);  
        }  
  
        return pq.poll();  
    }  
  
    public static void printHuffmanTree(HuffmanNode root, String code) {  
        if (root == null) return;  
  
        if (root.value != '-') {  
            System.out.println(root.value + ": " + code);  
        }  
    }  
}
```

```

        printHuffmanTree(root.left, code + "0");
        printHuffmanTree(root.right, code + "1");
    }

    public static void main(String[] args) {
        char[] values = {'A', 'B', 'C', 'D', 'E', 'F'};
        int[] frequencies = {5, 25, 7, 15, 4, 12};

        HuffmanNode root = buildHuffmanTree(values, frequencies);

        System.out.println("Huffman Codes:");
        printHuffmanTree(root, "");
    }
}

```

5. OUTPUT_2

Output:

Huffman Codes:

F: 00

D: 01

C: 100

E: 1010

A: 1011

B: 11

9. SUMMARY:

The Knapsack Problem and Huffman Coding Algorithm are fundamental algorithms in computer science used for optimization and data compression. The Knapsack

Problem maximizes the total value of items in a knapsack without exceeding its weight capacity, while Huffman Coding assigns variable-length codes to input characters based on their frequencies.